WELT 3 · DOPPELSTUNDEN 21 UND 22 · IDE-LEVEL 3.3

W3·3 – Die Schatzkammer: Teile und herrsche

Datei: `AQ_W3_L3_Labyrinth_Geruest.txt` (Klasse TeileUndHerrsche, Level 3.3) · Im Lab: **Level 3.5** und **Level 3.6** · Teil 1 in DS 21, Teil 2 in DS 22

Im Herzen des Labyrinths liegt die **Schatzkammer**: Hunderte Truhen, jede mit einer Zahl für ihren Goldwert. Die Zwerge müssen schnell die wertvollste Truhe finden, die Truhen sortieren und gezielt nach einem Wert suchen. Ein einzelner Zwerg, der alle Truhen nacheinander vergleicht, braucht zu lange. Die Zwerge haben eine bessere Idee: **Teile den Haufen, lass jede Hälfte von einem Gehilfen erledigen und setze die Ergebnisse zusammen** – und jeder Gehilfe macht es genauso.

Dieses Prinzip heißt **Teile und herrsche** (lateinisch *divide et impera*). Es steckt in den schnellen Such- und Sortierverfahren und kam im Abitur mehrfach vor: als Erläuterung an einem selbst gewählten Beispiel, bei Mergesort und Quicksort und bei unbekannten rekursiven Sortierverfahren, deren Ablauf dokumentiert werden musste.

Teil 1 – Teilen, herrschen, zusammenführen (DS 21)

Infoteil 1: Das Prinzip Teile und herrsche

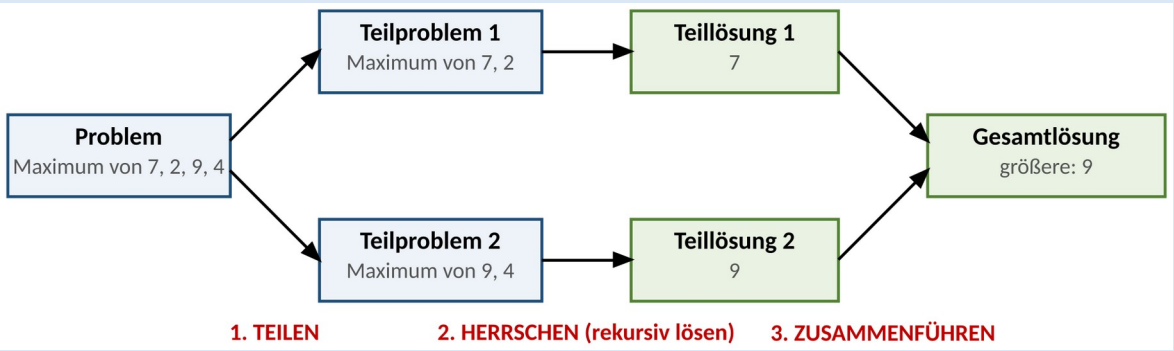


Abbildung 1: Die drei Schritte am Beispiel „Maximum von vier Truhen“

1. **Teilen:** Das Problem wird in kleinere Teilprobleme **derselben Art** zerlegt (meist zwei Hälften).
2. **Herrschen:** Die Teilprobleme werden **rekursiv** gelöst – mit demselben Verfahren. Sehr kleine Teilprobleme (ein Element, leerer Bereich) werden direkt gelöst: der **Basisfall**.
3. **Zusammenführen:** Aus den Teillösungen wird die Lösung des ganzen Problems gebaut.

Teilbereiche einer Reihung beschreibt man mit **Indexgrenzen**: `von … bis` bzw. `lo … hi` (beide Grenzen gehören dazu). So muss nichts kopiert werden. Die Mitte ist `mitte ← (von + bis) / 2` (ganzzahlige Division).

Aufgabe 0: Das Spielkarten-Team

PAPIER ★

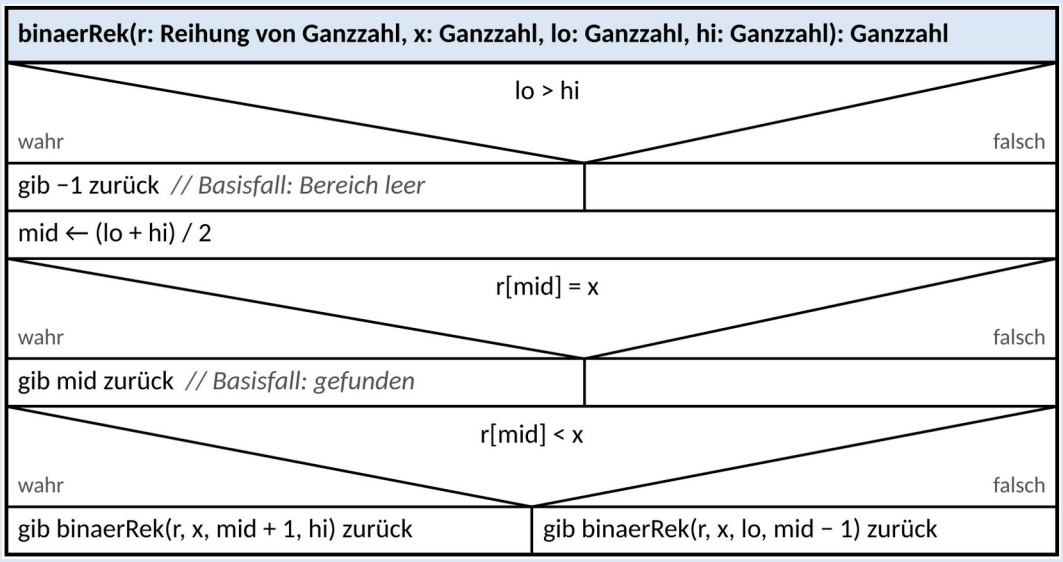
Jedes Team bekommt 8 Spielkarten einer Farbe, gemischt. **Spielregel:** Wer einen Stapel mit mehr als einer Karte bekommt, halbiert ihn und gibt die Hälften an zwei andere Personen weiter (= zwei Selbstaufrufe). Wer eine einzelne Karte bekommt, gibt sie sofort zurück (= Basisfall). Wer zwei sortierte Stapel zurückbekommt, **verschmilzt** sie: immer die kleinere der beiden obersten Karten nehmen.

a) Wie viele Aufträge (weitergegebene Stapel, einschließlich der Einzelkarten und des ersten Stapels) gibt es bei 8 Karten insgesamt? **[1 BE]**

b) Beim Verschmelzen zweier sortierter Stapel mit je 4 Karten: Wie viele Vergleiche sind es mindestens, wie viele höchstens? Gib für beide Fälle ein Beispiel an. **[2 BE]**

Infoteil 2: Binäre Suche – rekursiv

Die binäre Suche aus Welt 2 ist Teile und herrsche in Reinform: Vergleiche mit dem mittleren Element, dann suche **nur in einer Hälfte** weiter. Rekursiv formuliert:



- Zwei Basisfälle: Der Bereich ist **leer** (lo > hi) → −1, oder der Wert ist **gefunden** → mid.
- Der Bereich wird in jedem Schritt mindestens um die Hälfte kleiner – deshalb `mid + 1` bzw. `mid − 1`, nicht mid.
- „Zusammenführen" ist hier trivial: Das Ergebnis des Selbstaufrufs wird unverändert zurückgegeben (Endrekursion).

Aufgabe 1: Das Orakel der Schatzkammer

PAPIER ★★

Die Truhen sind aufsteigend nach Gold sortiert:

Index	0	1	2	3	4	5	6	7	8	9	10
r	4	9	13	21	26	30	38	45	51	60	72

a) Dokumentiere `binaerRek(r, 45, 0, 10)` und `binaerRek(r, 10, 0, 10)` jeweils in einer Tabelle. **[4 BE]**

Tiefe *)	lo	hi	mid	r[mid]	Rückgabe
0					

Tiefe *)	lo	hi	mid	r[mid]	Rückgabe
0					

links: x = 45 · rechts: x = 10

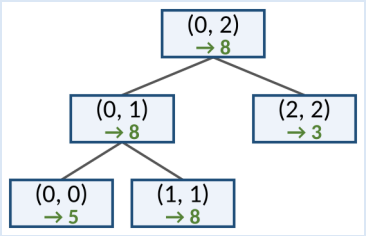
**) Man spricht bei einem rekursiven Aufruf davon, dass sich die Rekursionstiefe um 1 erhöht. Der Erstaufruf hat die Rekursionstiefe 0.*

b) Gib an, wie viele Aufrufe von binaerRek bei dieser Reihung **höchstens** vorkommen können (Erstaufruf mitgezählt). Schätze die Höchstzahl für 1000 und für 1 000 000 Truhen ab und begründe. **[3 BE]**

c) Erläutere an der binären Suche die drei Schritte Teilen, Herrschen und Zusammenführen. **[2 BE]**

Infoteil 3: Maximum nach Teile und herrsche – der Teilungsbaum

```
public static int maximumTH(int[] r, int von, int bis) {
    if (von == bis) {
        return r[von];                // Basisfall
    }
    int mitte = (von + bis) / 2;      // TEILEN
    int links = maximumTH(r, von, mitte);
    int rechts = maximumTH(r, mitte + 1, bis);
    if (links > rechts) {             // ZUSAMMENFÜHREN
        return links;
    }
    return rechts;
}
```



Teilungsbaum für r = {5, 8, 3}: (von, bis) und Rückgabe

- Jeder Knoten des **Teilungsbaums** ist ein Aufruf, beschriftet mit seinem Bereich (von, bis). Die linke Hälfte ist von … mitte, die rechte mitte + 1 … bis – bei ungerader Länge ist die **linke** Hälfte die größere.
- Die Blätter sind die Basisfälle (ein Element). Die Rückgaben wandern von unten nach oben.

Aufgabe 2: Die wertvollste Truhe

PAPIER ★★

Gegeben ist r = {6, 3, 8, 5, 9, 2} (Indizes 0 bis 5).

a) Zeichne den Teilungsbaum für `maximumTH(r, 0, 5)` mit (von, bis) und Rückgabewert an jedem Knoten. **[3 BE]**

b) Gib die Anzahl der Aufrufe, die größte Rekursionstiefe und die Anzahl der Vergleiche `links > rechts` an. **[2 BE]**

c) Begründe, dass maximumTH für n Elemente immer genau 2n − 1 Aufrufe auslöst. **[2 BE]**

d) Ein Zwerg meint: „Teile und herrsche ist immer schneller als eine Schleife." Vergleiche die Anzahl der Vergleiche von maximumTH mit der einer Schleife, die das Maximum sucht, und nimm Stellung. **[2 BE]**

Infoteil 4: Mergesort – Sortieren durch Verschmelzen

1. **Basisfall:** Eine Reihung mit höchstens einem Element ist sortiert – es wird eine Kopie geliefert.
2. **Teilen:** `mitte ← länge / 2`; links = r[0 … mitte − 1], rechts = r[mitte … länge − 1] (mit `teilstueck`, bis exklusiv).
3. **Herrschen:** beide Hälften rekursiv mit Mergesort sortieren.
4. **Zusammenführen:** `verschmelze` – das Reißverschluss-Verfahren aus Welt 0: immer das kleinere der beiden vordersten Elemente übernehmen, bei Gleichstand das aus der **linken** Hälfte; ist eine Hälfte leer, den Rest der anderen anhängen.

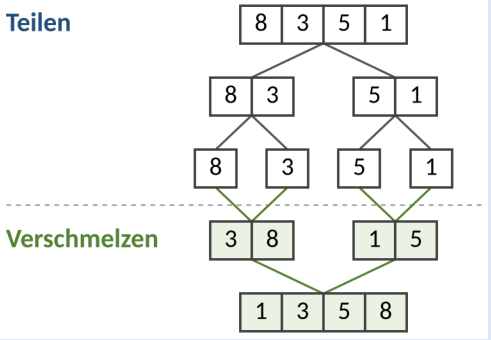


Abbildung 2: Teilungsbaum (oben) und Verschmelzungsbaum (unten) für {8, 3, 5, 1}. Im Teilungsbaum stehen die Teilreihungen beim Aufruf, im Verschmelzungsbaum die sortierten Rückgaben.

Arbeit: Jede **Ebene** fasst jedes Element einmal an (etwa n Vergleiche); es gibt etwa $\log_2 n$ Ebenen. Zusammen: etwa $\mathbf{n \cdot \log_2 n}$ Vergleiche – auch im schlechtesten Fall.

Aufgabe 3: Truhen teilen, Truhen verschmelzen

PAPIER ★★

Gegeben ist r = {21, 8, 34, 13, 5, 17}.

a) Zeichne den Teilungs- und den Verschmelzungsbaum von `mergeSort(r)`. **[4 BE]**

b) Dokumentiere den letzten Aufruf `verschmelze({8, 21, 34}, {5, 13, 17})` in einer Tracetabelle. i, j und k sind die Positionen in a, b und im Ergebnis. Eine Zeile vor der Schleife (Startwerte), danach eine Zeile nach jedem übernommenen Element – auch beim Anhängen des Rests. **[3 BE]**

i	j	k	Ergebnis bis jetzt
0			

c) Gib an, wie oft mergeSort und wie oft verschmelze aufgerufen werden und wie viele Vergleiche insgesamt nötig sind. **[2 BE]**

d) Die Truhen (3, a), (1, b), (3, c), (2, d) werden mit Mergesort nach der Zahl sortiert. Gib das Ergebnis an, wenn verschmelze mit ≤ vergleicht, und wenn es mit < vergleicht. Erläutere damit den Begriff **stabil**. **[2 BE]**

Aufgabe 4: Level 3.3 – Basis und Erweiterung

IDE ★ – ★★

Arbeite in der Klasse **TeileUndHerrsche**, Testklasse `L33TeileHerrscheTest`.

Stufe	Methoden (Signatur wie im Abitur)	Tests beginnen mit
★ Basis	binaerRek(r: Reihung von Ganzzahl, x: Ganzzahl, lo: Ganzzahl, hi: Ganzzahl): Ganzzahl (REKURSIV) binaer(r: Reihung von Ganzzahl, x: Ganzzahl): Ganzzahl	`basis_`
★★ Erweiterung	maximumTH(r: Reihung von Ganzzahl, von: Ganzzahl, bis: Ganzzahl): Ganzzahl (REKURSIV) teilstueck(r, von, bis): Reihung von Ganzzahl · verschmelze(a, b): Reihung von Ganzzahl mergeSort(r: Reihung von Ganzzahl): Reihung von Ganzzahl (REKURSIV, NEUE Reihung)	`erweiterung_`

- `teilstueck(r, von, bis)`: **bis gehört nicht dazu**, die neue Reihung hat die Länge bis – von. Teste teilstueck und verschmelze einzeln, bevor du mergeSort schreibst.
- StackOverflowError bei mergeSort({2, 1})? Dann ist eine Hälfte leer und die andere so lang wie vorher – prüfe `mitte = r.length / 2`.

> 🎯 **Im Lab üben**
>
> - **Level 3.5 „Schatzkammer: Teile und herrsche“** – passt zu Teil 1, Aufgaben 0–4
>
> Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit Kurscode, Name und deinem persönlichen Code.

Teil 2 – Quicksort und unbekannte Verfahren (DS 22)

Infoteil 5: Quicksort – erst zerlegen, dann herrschen

Mergesort teilt stur in der Mitte und hat die Arbeit beim **Zusammenführen**. Quicksort macht es umgekehrt: Die Arbeit steckt im **Teilen**, das Zusammenführen ist geschenkt.

quickSort(r: Reihung von Ganzzahl, lo: Ganzzahl, hi: Ganzzahl): nichts	
lo < hi (wahr / falsch)	
p ← zerlege(r, lo, hi) *// TEILEN*	*// Basisfall*
quickSort(r, lo, p − 1) *// HERRSCHEN links*	
quickSort(r, p + 1, hi) *// HERRSCHEN rechts*	

zerlege(r: Reihung von Ganzzahl, lo: Ganzzahl, hi: Ganzzahl): Ganzzahl	
pivot ← r[lo]	
grenze ← lo	
für i von lo + 1 bis hi	
r[i] < pivot (wahr / falsch)	
grenze ← grenze + 1	
tausche r[grenze] und r[i]	
tausche r[lo] und r[grenze] *// Pivot an seinen Platz*	
gib grenze zurück	

- **zerlege** wählt das **linke** Element r[lo] als **Pivot**. `grenze` markiert das Ende des Bereichs der kleineren Werte. Jedes r[i] < pivot wird nach vorn getauscht. Zum Schluss kommt das Pivot an die Stelle grenze: Links davon stehen nur kleinere, rechts nur größere oder gleiche Werte – das Pivot steht an seiner **endgültigen** Position.
- **quickSort** sortiert danach nur noch die beiden Teile links und rechts vom Pivot rekursiv. Basisfall: Bereiche mit höchstens einem Element (lo ≥ hi).

Beispiel: `zerlege(r, 0, 4)` für r = {4, 7, 1, 5, 2}, Pivot 4 – 4 Vergleiche, Rückgabe 2:

i	grenze	Reihung danach	Tausch
–	0	4, 7, 1, 5, 2	
1	0	4, 7, 1, 5, 2	
2	1	4, 1, 7, 5, 2	tausche r[1] und r[2]
3	1	4, 1, 7, 5, 2	
4	2	4, 1, 2, 5, 7	tausche r[2] und r[4]
Ende	2	2, 1, 4, 5, 7	tausche r[0] und r[2]

Aufgabe 5: Der Torwächter zerlegt die Schar

PAPIER ★★

Gegeben ist r = {6, 9, 3, 7, 1, 8, 4, 2} (Indizes 0 bis 7).

a) Führe `zerlege(r, 0, 7)` Schritt für Schritt aus (Format wie im Beispiel): eine Zeile vor der Schleife (–), eine am Ende jedes Schleifendurchlaufs und eine nach dem Schlusstausch (Ende). Gib die Rückgabe und die Anzahl der Vergleiche an. **[4 BE]**

i	grenze	Reihung danach
–		
1		
2		
3		
4		
5		
6		
7		
Ende		

b) Dokumentiere den vollständigen Ablauf von `quickSort(r, 0, 7)`: eine Zeile für jeden Aufruf, in dem zerlegt wird (lo < hi), mit dem Zustand der Reihung **nach** zerlege. **[4 BE]**

Tiefe *)	lo	hi	p	Reihung nach zerlege	Vergleiche
0					

**) Man spricht bei einem rekursiven Aufruf davon, dass sich die Rekursionstiefe um 1 erhöht. Der Erstaufruf hat die Rekursionstiefe 0.*

c) Gib die Gesamtzahl der Vergleiche an. Wie viele Aufrufe von quickSort gibt es insgesamt (auch die mit lo ≥ hi)? **[2 BE]**

Aufgabe 6: Der schlechteste Tag des Torwächters

PAPIER ★★★

Ausgerechnet eine bereits **sortierte** Schar bringt den Torwächter ins Schwitzen.

a) Dokumentiere `quickSort(r, 0, 4)` für r = {1, 2, 3, 4, 5} wie in Aufgabe 5 b) und gib die Gesamtzahl der Vergleiche an. **[3 BE]**

Tiefe *)	lo	hi	p	Reihung nach zerlege	Vergleiche
0					

b) Begründe, dass Quicksort (Pivot links) für eine sortierte Reihung mit n Elementen n(n − 1)/2 Vergleiche braucht. Gib die größte Rekursionstiefe an, in der noch zerlegt wird. **[3 BE]**

c) Erläutere, warum dieser Fall für Mergesort kein Problem ist, und schlage eine Änderung der Pivotwahl vor, die den Fall entschärft. **[2 BE]**

Aufgabe 7: Transfer: Der Drittel-Golem

PAPIER ★★★

Ein Steingolem sortiert Truhen mit einem Verfahren, das niemand versteht. Die Operation `golemSort` (Abbildung 3) soll die Werte r[li] … r[re] aufsteigend sortieren. Die Division ist ganzzahlig.

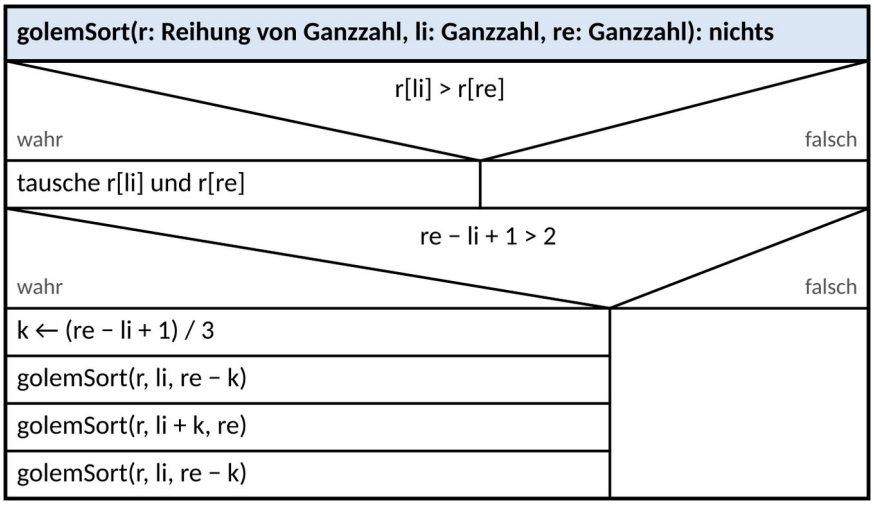


Abbildung 3: Struktogramm der rekursiven Operation golemSort

a) Dokumentieren Sie für r = {5, 8, 1, 3} den Aufruf `golemSort(r, 0, 3)` und die Selbstaufrufe mit **Rekursionstiefe 1**. Notieren Sie zu jedem Aufruf die Belegung der Reihung **zum Zeitpunkt des Aufrufs** und geben Sie die Belegung nach dem Ende des Erstaufrufs an. **[4 BE]**

Tiefe *)	Aufruf	r[0]	r[1]	r[2]	r[3]
0					

Belegung nach dem Ende des Erstaufrufs: r[0] = ____ r[1] = ____ r[2] = ____ r[3] = ____

**) Man spricht bei einem rekursiven Aufruf davon, dass sich die Rekursionstiefe um 1 erhöht. Der Erstaufruf hat die Rekursionstiefe 0.*

b) Ermitteln Sie die Anzahl aller Aufrufe von golemSort (einschließlich des Erstaufrufs) für n = 2, n = 3 und n = 4 Elemente. Für n = 6 sind es 40, für n = 9 bereits 121 Aufrufe. Beurteilen Sie golemSort im Vergleich zu mergeSort (2n − 1 Aufrufe). **[3 BE]**

c) Ein Zwerg schlägt vor, k ← (re − li + 1) / 2 zu verwenden, „dann wird es schneller“. Zeigen Sie an einem Beispiel mit vier Werten, dass golemSort dann nicht mehr korrekt sortiert, und erläutern Sie, welche Eigenschaft der Drittel-Teilung verloren geht. **[3 BE]**

d) Begründen Sie, dass golemSort für jede Eingabe endet. **[2 BE]**

Aufgabe 8: Der große Vergleich

PAPIER ★★

a) Fasse alle Sortierverfahren aus Welt 2 und Welt 3 in der Tabelle zusammen (aufsteigend sortieren, n = 8). Die erste Zeile ist ein Beispiel. **[5 BE]**

Verfahren	Vergleiche: schon sortiert	Vergleiche: absteigend sortiert	schlechtester Fall allgemein	zusätzlicher Speicher	stabil?
Selectionsort	28	28	n(n − 1)/2	keiner (in-place)	nein
Insertionsort					
Bubblesort (mit Abbruch)					
Mergesort					
Quicksort (Pivot links)					

b) Quicksort hat einen schlechteren schlechtesten Fall als Mergesort und wird trotzdem in der Praxis oft verwendet. Nenne zwei Gründe. **[2 BE]**

c) Wann ist Insertionsort trotz allem eine gute Wahl? Begründe. **[1 BE]**

Aufgabe 9: Level 3.3 Experte – Quicksort

IDE ★★★

Implementiere `zerlege(r: Reihung von Ganzzahl, lo: Ganzzahl, hi: Ganzzahl): Ganzzahl` nach Infoteil 5 (mit `vergleiche++` für jeden Vergleich r[i] < pivot) sowie `quickSort(r)` und `quickSort(r, lo, hi)`. Tests mit `experte_` in `L33TeileHerrscheTest`.

a) **Vorhersage ohne Gerät:** Das Hauptprogramm sortiert die Schätze {7, 3, 9, 2, 8, 5, 1, 6}. Sage voraus: Maximum (maximumTH), Index von 8 in der mit mergeSort sortierten Reihung und die Anzahl der Vergleiche von quickSort. Starte dann und vergleiche.

maximumTH	Index von 8 (binaer)	Vergleiche quickSort

b) **Code-Review im Team:** Sind binaerRek, maximumTH, mergeSort und quickSort rekursiv ohne Schleife? Steht im Basisfall von mergeSort eine **Kopie**? Ruft quickSort die Teile mit p − 1 und p + 1 auf?

Für Schnelle: Mehr aus der Schatzkammer

- **Quickselect:** Der k-kleinste Wert ohne vollständiges Sortieren: Nach zerlege steht das Pivot an Position p. Ist p = k, bist du fertig; sonst suchst du **nur in einem** Teil weiter. Implementiere `kKleinster(r, k)` rekursiv. Warum braucht es im Mittel nur etwa 2n Vergleiche?
- **Median aus drei:** Tausche vor dem Zerlegen den mittleren der Werte r[lo], r[(lo + hi) / 2], r[hi] nach vorn. Miss die Vergleiche für sortierte Reihungen mit n = 100.
- **Golem in der IDE:** Programmiere golemSort mit einem Aufrufzähler. Wie viele Aufrufe sind es für n = 27, 81? Stelle eine Vermutung über das Wachstum auf.

🎯 Im Lab üben

- **Level 3.6 „Torwächter: Quicksort und unbekannte Verfahren“** – passt zu Teil 2, Aufgaben 5–8

Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit Kurscode, Name und deinem persönlichen Code.

Abi-Check Teile und herrsche

- ☐ Erläutern an einem Beispiel: alle drei Schritte benennen – **Teilen**, rekursiv **Herrschen** (mit Basisfall) und **Zusammenführen**.
- ☐ Grenzen: mitte gehört zu genau **einer** Hälfte (von … mitte und mitte + 1 … bis); jeder Teilbereich ist echt kleiner.
- ☐ Rekursionstabellen: Belegung der Reihung **zum Zeitpunkt des Aufrufs** (vor dessen Änderungen), Geschwisteraufrufe mit derselben Tiefe.
- ☐ Aufwand begründen: Anzahl der Aufrufe bzw. Vergleiche in Abhängigkeit von n angeben – nicht nur „schnell“ oder „langsam“.
- ☐ Quicksort: schlechtester Fall bei sortierter Eingabe und Pivot links, n(n − 1)/2 Vergleiche.